

(PROGRESS REPORT)

Học phần: Operating Systems [Multimedia] - 20252

Mã số dự án (Project No): 25236

Tên dự án (Project title): Optimize pipe implementation to improve IPC throughput

Thiết kế (Design thinking): [Link Figma](#)

Quản lý tác vụ (Task management): [Link JIRA](#)

Thành viên (Members): Nguyễn Sỹ Tuấn , Lê Huy Sơn, Vũ Tiến Long

Giai đoạn báo cáo (Working period): Từ: 18/4/2026 Đến: 24/4/2026

1 Summary of latest progress

(Viết khoảng 200-300 từ để tóm tắt quá trình thực hiện trong giai đoạn này và tổng mức độ hoàn thành hiện tại)

Như đã nói ở tuần trước, chúng tôi đã đưa ra các tasks phải thực hiện trong tuần này là:

1. Viết chương trình ví dụ về pipe trong xv6.
2. Viết chương trình tính toán thông lượng của pipe với dung lượng đi vào là `total_bytes`, mỗi lần ghi đọc là `chunk_bytes`.

Trong tuần trước, chúng tôi đã tổng quát về các hàm trong file `pipe.c`. Tuy nhiên để hiểu một cách chi tiết về file `pipe.c` rõ hơn. Chúng tôi sẽ phân tích về các hàm trong file `pipe.c` (`pipealloc()`, `pipeclose()`, `piperead()`, `pipewrite()`) rồi mới viết chương trình ví dụ về pipe. Thông lượng trong pipe được tính bằng:

$$Throughput = \frac{DataTransfer}{TimeTransfer}$$

Với:

- + `DataTransfer`: Dữ liệu được truyền đi (byte)
- + `TimeTransfer`: Thời gian truyền hết dữ liệu đó (s)

2 Review of technical activities

2.1 Activity 1: Tìm hiểu chi tiết các hàm trong file pipe.c

2.1.1 pipealloc()

Hàm này phân bổ cấu trúc pipe vào 2 đối tượng tập tin, nhận đầu vào là con trỏ trỏ tới tập 2 đối tượng tập tin đó, trả về giá trị 0 nếu việc phân bổ thành công và -1 nếu không thành công.

Đoạn mã hàm pipealloc() như sau:

```
int pipealloc(struct file **f0, struct file **f1)
{
    // Khởi tạo một cấu trúc pipe *p
    struct pipe *p;

    // Khởi tạo giá trị ban đầu cho pipe là trống
    p = 0;
    // Con trỏ trỏ tới các file được gán ban đầu là 0 để không trỏ đến file
    nào
    *f0 = *f1 = 0;
    // Hàm filealloc() (nằm trong file kernel/file.c) nhận đầu vào là file và
    trả về con trỏ dẫn đến file nếu khởi tạo hành công và trả về 0 nếu thất
    bại.
    // Nếu khởi tạo thất bại một trong hai file hoặc cả 2 file thì có vấn
    đề, đi đến nhãn bad
    if((*f0 = filealloc()) == 0 || (*f1 = filealloc()) == 0)
        goto bad;
    // Hàm kalloc() (nằm trong file kalloc.c): khởi tạo một page 4096 bytes,
    trả về con trỏ nếu thành công và trả về 0 nếu thất bại.
    // Nếu khởi tạo không thành công thì đến nhãn bad
    if((p = (struct pipe*)kalloc()) == 0)
        goto bad;
    // Nếu cấp phát thành công trang và file, gán cho các trường readopen,
    writeopen, vị trí viết == vị trí đọc == 0
    p->readopen = 1;
    p->writeopen = 1;
    p->nwrite = 0;
    p->nread = 0;
    // Khởi tạo khóa liên kết với pipe bằng hàm initlock() (nằm trong file
    initlock.h)
    initlock(&p->lock, "pipe");
```

```

// Gán giá trị cho các file đọc. ghi
// Một struct file (trong file file.c) gồm các trường: type(FD_NONE,
FD_PIPE, FD_INODE), reference, writable, readable, struct pipe, struct
inode, off
(*f0)->type = FD_PIPE;
(*f0)->readable = 1;
(*f0)->writable = 0;
(*f0)->pipe = p;
(*f1)->type = FD_PIPE;
(*f1)->readable = 0;
(*f1)->writable = 1;
(*f1)->pipe = p;
return 0;

//PAGEBREAK: 20
// Nhấn bad dùng để giải phóng bộ nhớ cho trang bằng hàm kfree() (trong
file kernel/kalloc.c) và đóng các file f0, f1 bằng hàm fileclose() (trong
file kernel/file.c)
bad:
if(p)
    kfree((char*)p);
if(*f0)
    fileclose(*f0);
if(*f1)
    fileclose(*f1);
return -1;
}

```

2.1.2 pipeclose()

Thỉnh thoảng tiến trình người dùng sẽ gọi systemcall đóng để đóng pipe, loại bỏ con trỏ tới file và giải phóng nó. Trong pipe để đóng các file đọc/ghi này, ta loại bỏ các con trỏ từ file trỏ tới ống (writeopen, readopen). Đoạn mã cho hàm pipeclose() như sau:

```

void pipeclose(struct pipe *p, int writable)
{
    // 1: Closing Writend, 0: Closing readend
    // Lấy khóa cho đối tượng ống
    acquire(&p->lock);
    // Đóng việc ghi, bằng cách đóng con trỏ ghi trỏ tới đối tượng tệp

```

```

if(writable){
    p->writeopen = 0;
    // Đánh thức bất kỳ tiến trình đọc nào đang chờ
    wakeup(&p->nread);
    // Đóng việc đọc
} else {
    p->readopen = 0;
    // Đánh thức bất kỳ tiến trình ghi nào đang chờ
    wakeup(&p->nwrite);
}
// Nếu đóng cả việc đọc và ghi -> hoàn thành, trả lại khóa và giải phóng
// ống, nếu chưa thì chỉ trả lại khóa
if(p->readopen == 0 && p->writeopen == 0){
    release(&p->lock);
    kfree((char*)p);
} else
    release(&p->lock);
}

```

2.1.3 piperead()

Hàm piperead() nhận đầu vào là một con trỏ tới đối tượng pipe, một con trỏ tới địa chỉ(addr) và một số byte mà ta muốn ghi(n). Trả về i là số byte đã đọc và nếu việc đọc thất bại, trả về 1. Đoạn mã cho hàm piperead() như sau:

```

int piperead(struct pipe *p, char *addr, int n)
{
    int i;
    // Lấy khóa.
    acquire(&p->lock);
    // Check xem pipe có empty và có tiến trình nào đang ghi hay không, nếu
    // có thì chuyển sang chế độ ngủ cho đến khi không còn trình ghi nữa.
    while(p->nread == p->nwrite && p->writeopen){ //DOC: pipe-empty
        // Check xem tiến trình có bị kết liễu hay chưa, nếu bị kết liễu, việc
        // ghi thất bại, trả về 1.
        if(myproc()->killed){
            release(&p->lock);
            return -1;
        }
        // Chế độ ngủ
    }
}

```

```

    sleep(&p->nread, &p->lock); //DOC: piperead-sleep
}
// Đọc dữ liệu từ buffer
for(i = 0; i < n; i++){ //DOC: piperead-copy
    // Đang đọc mà hết dữ liệu, dừng
    if(p->nread == p->nwrite)
        break;
    // Đọc dữ liệu từ bộ đệm
    addr[i] = p->data[p->nread++ % PIPESIZE];
}
// Đọc xong, đánh thức tiến trình ghi, trả khóa
wakeup(&p->nwrite); //DOC: piperead-wakeup
release(&p->lock);
return i;
}

```

2.1.4 pipewrite()

Hàm này cũng giống với piperead(), chỉ khác là nó ghi dữ liệu. Đoạn mã cho hàm pipewrite() như sau:

```

//PAGEBREAK: 40
int pipewrite(struct pipe *p, char *addr, int n)
{
    int i;
    // Lấy khóa
    acquire(&p->lock);
    // Ghi dữ liệu từ bộ đệm
    for(i = 0; i < n; i++){
        // Bộ nhớ đầy, cũng check như piperead(), nếu tiến trình bị kết liễu hoặc không cho phép đọc thì việc ghi thất bại, trả về 1, còn không thì sẽ đánh thức tiến trình đọc và trình ghi chuyển sang chế độ ngủ
        while(p->nwrite == p->nread + PIPESIZE){ //DOC: pipewrite-full
            if(p->readopen == 0 || myproc()->killed){
                release(&p->lock);
                return -1;
            }
            wakeup(&p->nread);
            sleep(&p->nwrite, &p->lock); //DOC: pipewrite-sleep
        }
    }
}

```

```

    // Ghi dữ liệu từ bộ đệm
    p->data[p->nwrite++ % PIPESIZE] = addr[i];
}
// Ghi xong, trả khóa, đánh thức trình đọc để đọc tiếp
wakeup(&p->nread); //DOC: pipewrite-wakeup1
release(&p->lock);
return n;
}

```

2.1.5 Nhận xét

Cấu trúc pipe trong hệ điều hành xv6 hoạt động trên 4 hàm đơn giản: *pipealloc()*, *piperead()*, *pipewrite()*, *pipeclose()*.

Ở phía người dùng, để gọi được các hàm này ta phải dùng các system call, ví dụ ta muốn phân bổ một pipe p, hàm *pipe(p)* trong không gian người dùng file user/uses.S sẽ thực thi đoạn mã

```

li a7, SYS_pipe
ecall
ret

```

Khi *ecall* được thực thi, phần cứng CPU sẽ nhảy đến một địa chỉ cố định trong Kernel mà hệ điều hành đã thiết lập sẵn.

- + kernel/trampoline.S: Chứa hàm *uservec*. Nó lưu lại trạng thái (các thanh ghi) của tiến trình người dùng để sau này còn quay lại được.
- + kernel/trap.c: Hàm *usertrap()* được gọi. Nó nhận ra đây là một lời gọi hệ thống (system call) và gọi tiếp hàm *syscall()*

syscall() đọc số *syscall* từ *a7*, tra bảng *syscalls[]*, gọi *sys_pipe*, *sys_pipe* ở kernel/sysfile.c sẽ gọi *pipealloc(&rfd, &wfd)* để tạo đối tượng pipe trong kernel/pipe.c

Ta thấy mỗi hàm trong file *pipe.c* đều cần có khóa, bằng các hàm *acquire()* và *release()*. Phương pháp này chúng tôi đã được học trên lớp, gọi là mutex locks, giúp ngăn chặn race condition và bảo vệ critical section.

Cơ chế mutex locks: Một tiến trình phải chiếm được khóa (acquire) trước khi vào critical section và giải phóng khóa (release) khi ra khỏi critical section.

```
while(true){
    acquire lock
        critical section
    release lock
        remainder section
}
```

Xv6 sử dụng spin-lock, giảm sự lãng phí tài nguyên CPU.

2.2 Activity 2: Viết chương trình tìm hiểu về pipe trong xv6

Đề bài (mục tiêu) của chương trình có thể mô tả như sau: Viết một chương trình user-space trong xv6 để minh họa cơ chế hoạt động của pipe IPC.

Chương trình phải:

- 1 tạo pipe bằng pipe(p)
- 2 fork() thành parent/child,
- 3 parent ghi dữ liệu qua p[1]
- 4 child đọc dữ liệu từ p[0]
- 5 đóng đúng các đầu pipe không dùng (close(p[0]),close(p[1]))
- 6 kiểm tra EOF khi writer đóng.
- 7 In log rõ ràng từng bước để quan sát luồng thực thi.

2.3 Activity 3: Viết chương trình tính thông lượng IPC

Công thức tính thông lượng như đã giới thiệu ở mục 1. Để tính thông lượng thì ta định nghĩa một số biến:

DEFAULT_TOTAL_BYTE: độ lớn (byte) đưa vào pipe

DEFAULT_CHUNK_BYTE: kích thước (byte) mỗi lần đọc/ghi (phải nhỏ hơn hoặc bằng bộ đệm)

TICKS_PER_SEC: do xv6 hoạt động theo ticks, ta cần định nghĩa 1 tick có giá trị bao nhiêu giây, do đó quy đổi được về giây

elapsed_tick: số lượng ticks để truyền hết DEFAULT_TOTAL_BYTE

Thời gian truyền hết sẽ là: $\text{DEFAULT_TOTAL_BYTE} / \text{elapsed_tick} / \text{TICKS_PER_SEC}$.

3 Personal contribution

Member 1 (Nguyen Sy Tuan): Ghi nhận xét, viết chương trình

Member 2 (Le Huy Son): Tìm hiểu chi tiết các hàm

Member 3 (Vu Tien Long): Tìm hiểu chi tiết các hàm

4 Developed code review

4.1 Algorithmic development

(Mô tả các thuật toán đã phát triển)

4.2 Source code enhanced and/or revised

(Các đoạn code đã được cải tiến hoặc chỉnh sửa) kernel/pipe.c:

```
#include "types.h"
#include "riscv.h"
#include "defs.h"
#include "param.h"
#include "spinlock.h"
#include "proc.h"
#include "fs.h"
#include "sleeplock.h"
#include "file.h"

#define PIPESIZE (PGSIZE - sizeof(struct spinlock) - sizeof(uint) * 2 -
sizeof(int) * 2)
#define PIPE_TRACE 0
#define PIPE_TRACE_MAX_EVENTS 1200
#define PIPE_CALL_TRACE 1

struct pipe {
    struct spinlock lock;
    char data[PIPESIZE];
    uint nread;    // number of bytes read
    uint nwrite;   // number of bytes written
    int readopen;  // read fd is still open
    int writeopen; // write fd is still open
```



```

};

static int pipe_trace_events = 0;

static void
pipe_trace(struct pipe *pi, char *event, int pid, int slot, int byteval)
{
    int used;
    if(PIPE_TRACE == 0)
        return;
    if(pipe_trace_events >= PIPE_TRACE_MAX_EVENTS)
        return;

    used = pi->nwrite - pi->nread;
    printf("pipe-trace[%d] %s pid=%d nread=%d nwrite=%d used=%d free=%d",
        pipe_trace_events,
        event,
        pid,
        (int)pi->nread,
        (int)pi->nwrite,
        used,
        (int)PIPESIZE - used);
    if(slot >= 0)
        printf(" slot=%d", slot);
    if(byteval >= 0)
        printf(" byte=0x%x", byteval & 0xff);
    printf("\n");

    pipe_trace_events++;
    if(pipe_trace_events == PIPE_TRACE_MAX_EVENTS)
        printf("pipe-trace: reached max events (%d), suppressing further\n",
            PIPE_TRACE_MAX_EVENTS);
}

int
pipealloc(struct file **f0, struct file **f1)
{
    struct pipe *pi;

    pi = 0;

```

```

*f0 = *f1 = 0;
if((*f0 = filealloc()) == 0 || (*f1 = filealloc()) == 0)
    goto bad;
if((pi = (struct pipe*)kalloc()) == 0)
    goto bad;
pi->readopen = 1;
pi->writeopen = 1;
pi->nwrite = 0;
pi->nread = 0;
initlock(&pi->lock, "pipe");
(*f0)->type = FD_PIPE;
(*f0)->readable = 1;
(*f0)->writable = 0;
(*f0)->pipe = pi;
(*f1)->type = FD_PIPE;
(*f1)->readable = 0;
(*f1)->writable = 1;
(*f1)->pipe = pi;
//if(PIPE_CALL_TRACE){
    //printf("pipe-call: pipealloc() pid=%d pi=%p pipesize=%d\n",
        //    myproc()->pid, pi, (int)PIPESIZE);
//}
pipe_trace(pi, "pipealloc", myproc()->pid, -1, -1);
return 0;

bad:
if(pi)
    kfree((char*)pi);
if(*f0)
    fileclose(*f0);
if(*f1)
    fileclose(*f1);
return -1;
}

void
pipeclose(struct pipe *pi, int writable)
{
    int pid;
    pid = myproc()->pid;
    //if(PIPE_CALL_TRACE){

```

```

        //printf("pipe-call: pipeclose() pid=%d writable=%d pi=%p\n",
            //    pid, writable, pi);
    //}
    acquire(&pi->lock);
    if(writable){
        pi->writeopen = 0;
        pipe_trace(pi, "pipeclose(write-end)", pid, -1, -1);
        wakeup(&pi->nread);
    } else {
        pi->readopen = 0;
        pipe_trace(pi, "pipeclose(read-end)", pid, -1, -1);
        wakeup(&pi->nwrite);
    }
    if(pi->readopen == 0 && pi->writeopen == 0){
        release(&pi->lock);
        kfree((char*)pi);
    } else
        release(&pi->lock);
}

int
pipewrite(struct pipe *pi, uint64 addr, int n)
{
    int i = 0;
    struct proc *pr = myproc();

    //if(PIPE_CALL_TRACE){
        //printf("pipe-call: pipewrite() pid=%d n=%d pi=%p\n", pr->pid, n,
            //    pi);
    //}

    acquire(&pi->lock);
    pipe_trace(pi, "pipewrite(begin)", pr->pid, -1, -1);
    while(i < n){
        if(pi->readopen == 0 || killed(pr)){
            pipe_trace(pi, "pipewrite(abort:reader-closed-or-killed)", pr->pid,
                -1, -1);
            release(&pi->lock);
            return -1;
        }
        if(pi->nwrite == pi->nread + PIPESIZE){ //DOC: pipewrite-full

```

```

    pipe_trace(pi, "pipewrite(full->sleep)", pr->pid, -1, -1);
    wakeup(&pi->nread);
    sleep(&pi->nwrite, &pi->lock);
    pipe_trace(pi, "pipewrite(wakeup)", pr->pid, -1, -1);
} else {
    char ch;
    int slot;
    if(copyin(pr->pagetable, &ch, addr + i, 1) == -1)
        break;
    slot = pi->nwrite % PIPESIZE;
    pi->data[slot] = ch;
    pi->nwrite++;
    pipe_trace(pi, "pipewrite(byte)", pr->pid, slot, (uchar)ch);
    i++;
}
}
pipe_trace(pi, "pipewrite(done)", pr->pid, -1, -1);
wakeup(&pi->nread);
release(&pi->lock);

//if(PIPE_CALL_TRACE){
//    printf("pipe-call: pipewrite() pid=%d wrote=%d\n", pr->pid, i);
//}
return i;
}

int
piperead(struct pipe *pi, uint64 addr, int n)
{
    int i;
    struct proc *pr = myproc();
    char ch;

    //if(PIPE_CALL_TRACE){
    //    printf("pipe-call: piperead() pid=%d n=%d pi=%p\n", pr->pid, n,
    //        pi);
    //}

    acquire(&pi->lock);
    pipe_trace(pi, "piperead(begin)", pr->pid, -1, -1);
    while(pi->nread == pi->nwrite && pi->writeopen){ //DOC: pipe-empty

```

```

    if(killed(pr)){
        pipe_trace(pi, "piperead(abort:killed)", pr->pid, -1, -1);
        release(&pi->lock);
        return -1;
    }
    pipe_trace(pi, "piperead(empty->sleep)", pr->pid, -1, -1);
    sleep(&pi->nread, &pi->lock); //DOC: piperead-sleep
    pipe_trace(pi, "piperead(wakeup)", pr->pid, -1, -1);
}
for(i = 0; i < n; i++){ //DOC: piperead-copy
    int slot;
    if(pi->nread == pi->nwrite)
        break;
    slot = pi->nread % PIPESIZE;
    ch = pi->data[slot];
    if(copyout(pr->pagetable, addr + i, &ch, 1) == -1) {
        if(i == 0)
            i = -1;
        break;
    }
    pi->nread++;
    pipe_trace(pi, "piperead(byte)", pr->pid, slot, (uchar)ch);
}
pipe_trace(pi, "piperead(done)", pr->pid, -1, -1);
wakeup(&pi->nwrite); //DOC: piperead-wakeup
release(&pi->lock);
//if(PIPE_CALL_TRACE){
    //printf("pipe-call: piperead() pid=%d read=%d\n", pr->pid, i);
//}
    return i;
}

```

user/pipeexample.c:

```

include "kernel/types.h"
#include "user/user.h"

int
main(void)

```

```

{
    int p[2];
    int pid;
    char buf[64];
    int n;
    char *m1 = "message 1 from parent\n";
    char *m2 = "message 2 from parent\n";
    char *m3 = "message 3 from parent\n";

    printf("[user] step 1: call pipe(p)\n");

    if(pipe(p) < 0){
        fprintf(2, "pipeexample: pipe failed\n");
        exit(1);
    }
    printf("[user] pipe() success: p[0]=%d (read end), p[1]=%d (write end)\n", p[0], p[1]);

    printf("[user] step 2: fork()\n");
    pid = fork();
    if(pid < 0){
        fprintf(2, "pipeexample: fork failed\n");
        close(p[0]);
        close(p[1]);
        exit(1);
    }

    if(pid == 0){
        printf("[child] start: close write end p[1]=%d\n", p[1]);
        close(p[1]);
        while((n = read(p[0], buf, sizeof(buf) - 1)) > 0){
            buf[n] = 0;
            printf("[child] read %d bytes from p[0]=%d: %s", n, p[0], buf);
        }
        printf("[child] read returned %d -> EOF (writer closed)\n", n);
        printf("[child] close read end p[0]=%d\n", p[0]);
        close(p[0]);
        exit(0);
    }

    printf("[parent] start: close read end p[0]=%d\n", p[0]);

```

```

close(p[0]);

n = write(p[1], m1, strlen(m1));
printf("[parent] write #1 -> %d bytes to p[1]=%d\n", n, p[1]);
n = write(p[1], m2, strlen(m2));
printf("[parent] write #2 -> %d bytes to p[1]=%d\n", n, p[1]);
n = write(p[1], m3, strlen(m3));
printf("[parent] write #3 -> %d bytes to p[1]=%d\n", n, p[1]);

printf("[parent] close write end p[1]=%d\n", p[1]);
close(p[1]);

printf("[parent] wait child\n");
wait(0);
printf("[parent] done\n");
exit(0);
}

```

user/pipetest.c

```

#include "kernel/types.h"
#include "user/user.h"

#define DEFAULT_TOTAL_BYTES (8 * 1024 * 1024)
#define DEFAULT_CHUNK_BYTES 256
#define TICKS_PER_SEC 10

static int
min(int a, int b)
{
    if(a < b)
        return a;
    return b;
}

int
main(int argc, char *argv[])
{
    int p[2];
    int total_bytes = DEFAULT_TOTAL_BYTES;

```

```

int chunk_bytes = DEFAULT_CHUNK_BYTES;
char *buf;
int pid;
int start_ticks, end_ticks, elapsed_ticks;
int sent = 0;

if(argc >= 2)
    total_bytes = atoi(argv[1]);
if(argc >= 3)
    chunk_bytes = atoi(argv[2]);

if(total_bytes <= 0 || chunk_bytes <= 0){
    fprintf(2, "usage: pipetest [total_bytes] [chunk_bytes]\n");
    exit(1);
}

buf = malloc(chunk_bytes);
if(buf == 0){
    fprintf(2, "pipetest: malloc(%d) failed\n", chunk_bytes);
    exit(1);
}

memset(buf, 'a', chunk_bytes);

if(pipe(p) < 0){
    fprintf(2, "pipetest: pipe failed\n");
    exit(1);
}

pid = fork();
if(pid < 0){
    fprintf(2, "pipetest: fork failed\n");
    close(p[0]);
    close(p[1]);
    exit(1);
}

if(pid == 0){
    int got = 0;
    int n;
    close(p[1]);

```



```

while(got < total_bytes){
    n = read(p[0], buf, min(chunk_bytes, total_bytes - got));
    if(n <= 0)
        break;
    got += n;
}
close(p[0]);
exit(0);
}

close(p[0]);
start_ticks = uptime();
while(sent < total_bytes){
    int n = write(p[1], buf, min(chunk_bytes, total_bytes - sent));
    if(n <= 0){
        fprintf(2, "pipetest: write failed\n");
        close(p[1]);
        wait(0);
        exit(1);
    }
    sent += n;
}
close(p[1]);
wait(0);
end_ticks = uptime();

elapsed_ticks = end_ticks - start_ticks;
if(elapsed_ticks <= 0)
    elapsed_ticks = 1;

printf("IPC throughput benchmark\n");
printf("  total bytes : %d\n", total_bytes);
printf("  chunk bytes : %d\n", chunk_bytes);
printf("  elapsed      : %d ticks (~%d ms)\n",
        elapsed_ticks, elapsed_ticks * 100);
printf("  IPC throughput: %d B/s (%d KB/s)\n",
        (total_bytes * TICKS_PER_SEC) / elapsed_ticks,
        ((total_bytes * TICKS_PER_SEC) / elapsed_ticks) / 1024);

exit(0);
}

```

4.3 Testing result

(Kết quả kiểm thử, có thể chèn ảnh minh họa bên dưới)

pipetest.c:

```
$ pipetest
pipe-call: syscall() pid=3 num=4 -> sys_pipe()
pipe-call: sys_pipe() pid=3 user_fdarray=0x0x0000000000003f78
pipe-call: sys_pipe() pid=3 -> fd0=3 fd1=4 rf=0x0000000008001fc50 wf=0x0000000008001fc78
pipe-call: syscall() pid=3 <- sys_pipe() ret=0
trace: kernel/file.c -> filewrite() -> pipewrite()
trace: kernel/file.c -> fileread() -> piperead()
IPC throughput benchmark
  total bytes : 8388608
  chunk bytes : 256
  elapsed      : 182 ticks (~18200 ms)
  IPC throughput: 460912 B/s (450 KB/s)
```

pipeexample.c:

```
$ pipeexample
[user] step 1: call pipe(p)
pipe-call: syscall() pid=5 num=4 -> sys_pipe()
pipe-call: sys_pipe() pid=5 user_fdarray=0x0x0000000000003fa8
pipe-call: sys_pipe() pid=5 -> fd0=3 fd1=4 rf=0x0000000008001fc50 wf=0x0000000008001fc78
pipe-call: syscall() pid=5 <- sys_pipe() ret=0
[user] pipe() success: p[0]=3 (read end), p[1]=4 (write end)
[user] step 2: fork()
[p[achireld] nstart: close tw]r istart: close read teen end p[1]=d4 p
[0]=3
[p[archentild] ]writ eread #1 -> 222 b2ytes fro bm p[0ytes to] p[1]==34: m
essage [1 pfarom parentr
e[cnt] wrhite #ild] 2 r->ea 22d b 22 byyteste ts fromm p[0]=3 p:[1 ]mes=4
sa[pargee 2 frontm pa]r write e#3 ->n t
2[chi2 bytes tld] read 22 bytes fro pom[ 1p]=4
[[par0en]t] close write end =p[1]=43
: m[essage 3 froparm pareennt
t[ch] wilda] readt chil dr
eturned 0 -> EOF (writer closed)
[child] close read end p[0]=3
[parent] done
```

5 TODO

- List of remaining issues:**
- List of tasks to carry out in the next period:** Các task cần làm trong tuần sau: viết các task optimized cho pipe
- Milestone:** *(Cần làm gì thêm để đạt được mốc tiến độ tiếp theo?)*